# 矩陣運算與 FFT

本章從矩陣分塊與乘法出發，經消去、PA=LU 和最小平方，再以 DFT 與 FFT 求多項式卷積。

矩陣運算以分塊與因子分解組織計算；FFT 以偶奇拆分重用單位根。留意維度、主元、轉換慣例與補零長度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 矩陣運算把同一組係數重用

方程 $Ax = b$ 的重點是把係數與右端分開。固定 $A$，換 $b$ 時重用分解。全節採列向量、實數矩陣、$PA = LU$，$L$ 的對角線為 1。最小平方的殘差固定為 $r = b - A\hat{x}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 內維度相同才能相乘

$A \in \mathbb{R}^{m \times k}$，$B \in \mathbb{R}^{k \times n}$，則 $C = AB \in \mathbb{R}^{m \times n}$，$c_{ij} = \sum_{t=1}^{k} a_{it} b_{tj}$。每格做 $k$ 次乘法，合計 $mkn$ 次乘法與 $mn(k-1)$ 次加法。$AB$ 可定義不代表 $BA$ 可定義。

### 矩陣尺寸與一格內積

A：m × k

B：k × n

AB：m × n

共同索引 t = 1…k

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 逐格乘法保留索引的意義

取 $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$，$B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$。$c_{12} = 1(0) + 2(1) + 3(1) = 5$，故 $AB = \begin{bmatrix} 4 & 5 \\ 10 & 11 \end{bmatrix}$。$BA$ 是 $3 \times 3$，不能與 $AB$ 相等。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 分塊相乘仍須逐塊檢查維度

把內維度分成 $k = k_1 + k_2$。令 $A = [A_1\ A_2]$，$B = [B_1; B_2]$，其中 $A_s$ 為 $m \times k_s$，$B_s$ 為 $k_s \times n$。則 $AB = A_1B_1 + A_2B_2$。四象限分塊時 $C_{ij} = A_{i1}B_{1j} + A_{i2}B_{2j}$。區塊不能當可交換純量。

### 內維度切成 k₁ + k₂

A₁ : m × k₁

B₁ : k₁ × n

A₁B₁ + A₂B₂ : m × n

A₂ : m × k₂ ; B₂ : k₂ × n

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 分治減少乘法數才改變指數

一般正方分塊有 $8T(n/2) + \Theta(n^2) = \Theta(n^3)$。Strassen 以 7 個乘積重組四塊，得到 $7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7})$。可補零到二次冪再裁切。這是算術成本分析，未保證任意小矩陣更快。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Strassen：七個乘積如何構成？

將方陣分成同尺寸四塊 $A=[A_{11},A_{12};A_{21},A_{22}]$、$B=[B_{11},B_{12};B_{21},B_{22}]$，以七次區塊乘法計算：

$P_1=(A_{11}+A_{22})(B_{11}+B_{22})$；$P_2=(A_{21}+A_{22})B_{11}$；$P_3=A_{11}(B_{12}-B_{22})$；$P_4=A_{22}(B_{21}-B_{11})$。

$P_5=(A_{11}+A_{12})B_{22}$；$P_6=(A_{21}-A_{11})(B_{11}+B_{12})$；$P_7=(A_{12}-A_{22})(B_{21}+B_{22})$。區塊乘法不可交換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Strassen：重組與正確性

$C_{11}=P_1+P_4-P_5+P_7$，$C_{12}=P_3+P_5$；$C_{21}=P_2+P_4$，$C_{22}=P_1-P_2+P_3+P_6$。

例如展開 $C_{11}$，其餘交叉項相消，留下 $A_{11}B_{11}+A_{12}B_{21}$；其餘三塊同樣展開得到一般分塊乘法。每層七次半尺寸乘法與 $\Theta(n^2)$ 加減，遞迴式 $T(n)=7T(n/2)+\Theta(n^2)=\Theta(n^{\log_2 7})$，可補零至二次冪。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Strassen：2×2 數值追蹤

取 $A=\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix}$、$B=\begin{bmatrix}5 & 6\\ 7 & 8\end{bmatrix}$。七個純量乘積依序是 $P_1=65$、$P_2=35$、$P_3=-2$、$P_4=8$、$P_5=24$、$P_6=22$、$P_7=-30$。

重組得 $C_{11}=65+8-24-30=19$、$C_{12}=-2+24=22$、$C_{21}=35+8=43$、$C_{22}=65-35-2+22=50$，與直接相乘一致。這是算術次數分析；有限精度與較小規模的常數成本仍須另評估。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 消去先決定乘數再更新尾塊

在第 $k$ 步，以 $u_{kk}\neq 0$ 為 pivot，令 $l_{ik}=a_{ik}/u_{kk}$，執行 $R_i\leftarrow R_i-l_{ik}R_k$。消去後的上三角係數形成 $U$，乘數形成 $L$。分塊尾部更新是 Schur complement $A_{22}-a_{21}a_{12}/a_{11}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 可逆矩陣也可能有零 pivot

$A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$ 的行列式為 $-1$，但不換列的第一步會除以 0。交換兩列，$P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$，可得 $PA = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = IU$。零 pivot 只證明目前選擇失敗，不能直接宣告原矩陣奇異。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 第一欄換列與消去的完整數值

沿用 CLRS28.1 的 $A = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 4 & 4 \\ 5 & 6 & 3 \end{bmatrix}$。部分 pivoting 選欄內絕對值最大者，先 $R_1 \leftrightarrow R_3$ 得 $[5, 6, 3; 3, 4, 4; 1, 2, 0]$。乘數 $l_{21} = 3/5, l_{31} = 1/5$。更新 $R_2 \leftarrow R_2 - \frac{3}{5}R_1$、$R_3 \leftarrow R_3 - \frac{1}{5}R_1$ 後為 $[5, 6, 3; 0, 2/5, 11/5; 0, 4/5, -3/5]$。

第 1 輪：比較、換列、消去

A

$$\begin{bmatrix} 1 & 2 & 0 \\ 3 & 4 & 4 \\ 5 & 6 & 3 \end{bmatrix}$$

P₁A

$$\begin{bmatrix} 5 & 6 & 3 \\ 3 & 4 & 4 \\ 1 & 2 & 0 \end{bmatrix}$$

第 1 輪後工作矩陣

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 2/5 & 11/5 \\ 0 & 4/5 & -3/5 \end{bmatrix}$$

|1| < |3| < |5| → R₁ ↔ R₃

l₂₁ = 3/5, l₃₁ = 1/5; R₂ ← R₂ − (3/5)R₁; R₃ ← R₃ − (1/5)R₁

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 第二次換列也要交換已存乘數

第二欄 $4/5 > 2/5$，交換工作矩陣 $R_2, R_3$。同時交換 $L$ 已存的第一欄乘數，變成 $l_{21} = 1/5, l_{31} = 3/5$。此時工作矩陣為 $[5, 6, 3; 0, 4/5, -3/5; 0, 2/5, 11/5]$。用 $l_{32} = 1/2$ 消去後最後一格是 $11/5 - (1/2)(-3/5) = 5/2$。

第 2 輪：工作矩陣、P 與 L 同步換列

U₁

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 2/5 & 11/5 \\ 0 & 4/5 & -3/5 \end{bmatrix}$$

P₁

$$\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

L：已完成第 1 欄

$$\begin{bmatrix} 1 & 0 & 0 \\ 3/5 & 1 & 0 \\ 1/5 & 0 & 1 \end{bmatrix}$$

|4/5| > |2/5| → R₂ ↔ R₃

圖示為交換前；L 只交換已完成欄，未動對角線。



## 以 PA=LU 檢查分解慣例

$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$，$L = \begin{bmatrix} 1 & 0 & 0 \\ 1/5 & 1 & 0 \\ 3/5 & 1/2 & 1 \end{bmatrix}$，$U = \begin{bmatrix} 5 & 6 & 3 \\ 0 & 4/5 & -3/5 \\ 0 & 0 & 5/2 \end{bmatrix}$。兩邊相乘都等於 $[5, 6, 3; 1, 2, 0; 3, 4, 4]$。$P$ 作用於方程列，解未知量的順序不變。

消去後逐項核驗 PA = LU

U after R₂ ↔ R₃

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 4/5 & -3/5 \\ 0 & 2/5 & 11/5 \end{bmatrix}$$

U

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 4/5 & -3/5 \\ 0 & 0 & 5/2 \end{bmatrix}$$

PA = LU

$$\begin{bmatrix} 5 & 6 & 3 \\ 1 & 2 & 0 \\ 3 & 4 & 4 \end{bmatrix}$$

l₃₂ = (2/5)/(4/5) = 1/2; R₃ ← R₃ − (1/2)R₂

LU rows: (5, 6, 3); (1, 6/5+4/5, 3/5−3/5);

(3, 18/5+2/5, 9/5−3/10+5/2) = (3, 4, 4)

## 為何非奇異矩陣的 LUP 能持續選主元？

先看一個可逆的當前尾塊 $K$。其第一欄必有非零元素，否則 $K$ 有全零欄而奇異。交換尾塊中的列，把非零元素移到 pivot 位置 $a$；列交換不改變可逆性。

令換列後的尾塊為 $K' = PK$。分塊消去後 $\det K' = a \det S$，而 $\det K' = \pm \det K$。因 $\det K \neq 0$ 且 $a \neq 0$，新的 Schur complement $S$ 仍可逆，可重複相同步驟直至最後。後續換列時也須交換 $L$ 已儲存的對應乘數，最後得到 $PA = LU$。實數運算中選最大絕對值的 partial pivot 有助控制局部乘數，但不保證所有問題都數值穩定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 原地 LUP 的輸入、輸出與記憶體契約

輸入為實數 $n \times n$ 矩陣 $A$。工作陣列 $W$ 初始複製 $A$，排列向量 $p = [1, \ldots, n]$；最後 $W$ 的對角線及上三角存 $U$，嚴格下三角存 $L$ 的乘數，$L$ 的對角線隱含為 $1$。排列向量滿足 $(Pb)_i = b_{p_i}$，輸出關係是 $PA = LU$。

下列演算法採精確算術及一基索引；對 $n = 0$ 可約定輸出空分解。所謂「原地」指覆寫工作矩陣；若仍需保留原始 $A$，複製需另用 $\Theta(n^2)$ 空間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 原地 LUP：選主元、換整列、更新尾塊

對 $k = 1, \dots, n$，先在 $i = k, \dots, n$ 選使 $|W_{ik}|$ 最大的列 $q$。若 $W_{qk} = 0$，回報矩陣奇異並停止；否則交換 $W$ 的第 $k$ 與 $q$ 整列，也交換 $p_k, p_q$。

對每個 $i = k+1, \dots, n$，先存 $W_{ik} \leftarrow W_{ik}/W_{kk}$；再對 $j = k+1, \dots, n$ 更新 $W_{ij} \leftarrow W_{ij} - W_{ik}W_{kj}$。整列交換包含 $j < k$ 的既有乘數；只交換未處理的尾塊會破壞 $PA = LU$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 已完成區塊不變量與更新證明

完成前 $k$ 步後，將目前排列的原矩陣寫為

$PA = \begin{bmatrix} L_{11} & 0 \\ L_{21} & I \end{bmatrix} \begin{bmatrix} U_{11} & U_{12} \\ 0 & S \end{bmatrix}$，其中左上兩個三角塊皆為 $k \times k$，$S$ 是尚未消去的 Schur complement。已處理欄中的 $L_{11}, L_{21}$ 和已完成列中的 $U_{11}, U_{12}$ 都由 $W$ 保存。

下一步換列時，同步重排 $L_{21}$ 已儲存的舊乘數；把新 pivot 之下各值除以 pivot 得新乘數，再以外積更新 $S$。這正是矩陣相乘的尾塊公式，因而維持等式。初始 $k = 0$ 時 $S = A$；$k = n$ 時 $S$ 為空，得到 $PA = LU$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 零主元回報的意義與限制

精確算術下，若當前欄的剩餘元素全為 0，尾塊含零欄，故尾塊奇異；先前 pivot 均非零且排列、消去步驟可逆，所以原 $A$ 奇異。反之，若 $A$ 可逆，每步尾塊可逆，故總有非零候選，演算法完成。

在浮點算術中，用閾值把小 pivot 視為零，只表示按所選尺度與容忍值拒絕分解，不能直接證明數學上的矩陣奇異；結果還受捨入、縮放與條件數影響。若需數值秩，應使用合適的 rank-revealing 方法並說明閾值。

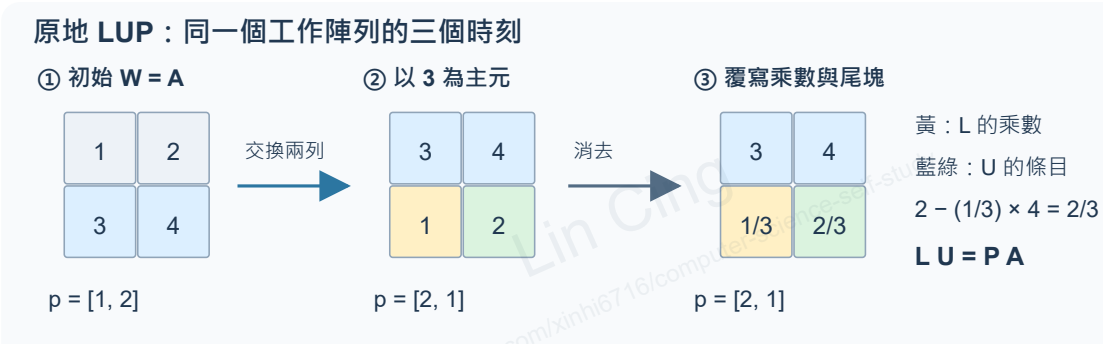
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 原地 LUP 追蹤：列交換與一個尾塊

取 $A = [1, 2; 3, 4]$、$b = (5, 11)^T$。第一欄最大絕對值為 3，交換兩列並設 $p = (2, 1)$，工作矩陣為 $[3, 4; 1, 2]$。

計算 $l_{21} = 1/3$，更新右下元素 $2 - (1/3)4 = 2/3$，所以 $W = [3, 4; 1/3, 2/3]$，它同時存 $L = [1, 0; 1/3, 1]$、$U = [3, 4; 0, 2/3]$。檢查 $LU = [3, 4; 1, 2] = PA$；$Pb = (11, 5)^T$。




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 前代依序得到中間右端

取 $b=(3,7,8)^T$，先排列成 $Pb=(8,3,7)^T$。解 $Ly=Pb$。$y_1=8$，$y_2=3-(1/5)8=7/5$，$y_3=7-(3/5)8-(1/2)(7/5)=3/2$。一般式 $y_i=(Pb)_i-\sum_{j<i}l_{ij}y_j$。每個右端前代為 $\Theta(n^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 回代從最後一列向上求解

解 $Ux=y$。$x_3=(3/2)/(5/2)=3/5$。$x_2=[7/5+(3/5)(3/5)]/(4/5)=11/5$。$x_1=[8-6(11/5)-3(3/5)]/5=-7/5$。代回原矩陣得 $Ax=(3,7,8)^T$。一般式 $x_i=(y_i-\sum_{j>i}u_{ij}x_j)/u_{ii}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 分解一次再處理多個右端

一般稠密消去的尾塊更新總量是 $\sum_{k=1}^{n-1}(n-k)^2=\Theta(n^3)$。已有分解時，每個新右端只需兩次三角解 $\Theta(n^2)$。$q$ 個右端總成本 $\Theta(n^3+qn^2)$。部分 pivoting 仍可能有元素成長，不是任意問題的精度保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 行列式與逆矩陣由分解導出

若共換列 $s$ 次，$\det A=(-1)^s\prod_i u_{ii}$。本例兩次換列，故 $\det A=5(4/5)(5/2)=10$。求逆解 $Az_j=e_j$，各 $z_j$ 是逆矩陣第 $j$ 欄。分解後 $n$ 次解共 $\Theta(n^3)$；單次 $Ax=b$ 直接解即可，先求逆多做了工作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 前代與回代的迴圈不變量

對單位下三角 $L$ 求 $Ly = Pb$：第 $i$ 步令 $y_i = (Pb)_i - \sum_{j<i} l_{ij} y_j$。進入第 $i$ 步前，$1, \dots, i-1$ 列已成立且這些 $y_j$ 不再改變；新式使第 $i$ 列成立，故歸納後 $Ly = Pb$。

對非奇異上三角 $U$ 求 $Ux = y$：由 $i = n$ 往 1 計算 $x_i = (y_i - \sum_{j>i} u_{ij} x_j)/u_{ii}$。進入第 $i$ 步時 $i+1, \dots, n$ 列成立；上三角使這些列不依賴 $x_i$，本步又使第 $i$ 列成立。非零對角主元保證除法成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 同一 LUP 例子的兩次三角解

前代：$y_1 = 11$，$y_2 = 5 - (1/3)11 = 4/3$。回代：$x_2 = (4/3)/(2/3) = 2$，$x_1 = (11 - 4 \cdot 2)/3 = 1$。最後 $A(1,2)^T = (5,11)^T = b$。

第 2 列回代先成立；計算 $x_1$ 時不改變 $x_2$，因此第 2 列仍成立。這就是回代不變量在數值例子中的具體作用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

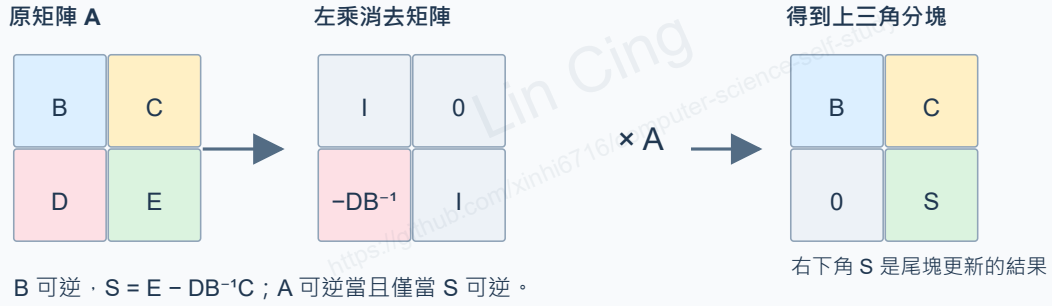
## 從消去看分塊反矩陣

令 $A=\begin{bmatrix}B & C\\ D & E\end{bmatrix}$，$B$ 為可逆方陣，其餘區塊的維度使乘法有意義。定義 Schur complement（舒爾補）$S=E-DB^{-1}C$。左乘分塊消去矩陣，可得

$\begin{bmatrix}I & 0\\ -DB^{-1} & I\end{bmatrix}A=\begin{bmatrix}B & C\\ 0 & S\end{bmatrix}$。

先消去左下區塊，再處理 $S$；區塊順序不可隨意交換。

**Schur complement：左下消去，右下跟著更新**




右下角 S 是尾塊更新的結果

B 可逆，S = E − DB⁻¹C；A 可逆當且僅當 S 可逆。



## 分塊反矩陣：公式與存在條件

若 $B$ 與 $S$ 都可逆，將分塊消去的三個因子逆序相乘，得到

$A^{-1}=\begin{bmatrix}B^{-1}+B^{-1}CS^{-1}DB^{-1} & -B^{-1}CS^{-1}\\ -S^{-1}DB^{-1} & S^{-1}\end{bmatrix}$。

實際做題時可直接檢查 $AA^{-1}=I$。在 $B$ 可逆的前提下，$A$ 可逆當且僅當 $S$ 可逆；若 $B$ 不可逆，這個公式不能使用，但不代表 $A$ 不可逆。

## 同一矩陣分塊求逆：先算 Schur complement

取 $A=\begin{bmatrix}4&2&2\\2&5&1\\2&1&3\end{bmatrix}$，切在前兩列與欄：$B=\begin{bmatrix}4&2\\2&5\end{bmatrix}$，$C=(2,1)^T$，$D=C^T$，$E=3$。

$B^{-1}=\frac{1}{16}\begin{bmatrix}5&-2\\-2&4\end{bmatrix}$，$B^{-1}C=(1/2,0)^T$，$S=3-(2,1)(1/2,0)^T=2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 同一矩陣分塊求逆：逐塊組裝

$S^{-1}=1/2$，右上塊 $-B^{-1}CS^{-1}=(-1/4,0)^T$，左下塊為其轉置。

左上塊 $B^{-1}+(1/2,0)^T(1/2)(1/2,0)=\begin{bmatrix}7/16&-1/8\\-1/8&1/4\end{bmatrix}$。

因此 $A^{-1}=\begin{bmatrix}7/16&-1/8&-1/4\\-1/8&1/4&0\\-1/4&0&1/2\end{bmatrix}$。例如第一列乘第一欄為 $4(7/16)+2(-1/8)+2(-1/4)=1$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 正定性保證遞迴不會遇到零主元

實對稱矩陣 $A$ 為正定（SPD），指每個非零 $z$ 都有 $z^TAz>0$。分塊 $A=\begin{bmatrix}B & C\\ C^T & D\end{bmatrix}$ 時，$B$ 也是 SPD：對非零 $x$，取 $z=(x,0)$，得 $x^TBx>0$。

$S=D-C^TB^{-1}C$ 同樣 SPD：對非零 $y$，取 $z=(-B^{-1}Cy,y)$，則 $y^TSy=z^TAz>0$。因此 $B,S$ 均可逆，能遞迴使用分塊公式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Cholesky 分解：平方根與尾塊

實對稱正定矩陣有唯一分解 $A=LL^T$，其中 $L$ 是對角線為正的下三角矩陣。把第一列欄寫成 $A=\begin{bmatrix}a & r^T\\ r & D\end{bmatrix}$：因 $a>0$，令 $\ell_{11}=\sqrt{a}$、$v=r/\sqrt{a}$，剩餘矩陣 $S=D-vv^T$ 是 SPD。

對 $S$ 遞迴分解 $S=L_2L_2^T$，便有 $L=\begin{bmatrix}\sqrt{a} & 0\\ v & L_2\end{bmatrix}$。每一步的平方根都為正；以相同對角線符號由第一欄遞迴決定，可證唯一。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Cholesky 的逐元素遞迴公式

設實對稱正定矩陣 $A = LL^T$，$L$ 為正對角下三角矩陣。由 $a_{ij} = \sum_{k=1}^{\min(i,j)} l_{ik} l_{jk}$，按欄 $j = 1, \dots, n$ 依序求：

$l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2}$；對 $i > j$，$l_{ij} = \dfrac{a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk}}{l_{jj}}$。

空和取 $0$。先求對角線，再求同欄下方元素；公式中的舊欄 $k < j$ 都已計算完成，上三角元素固定為 $0$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 為什麼平方根與除法都合法？

第 $j$ 步的根號內是前 $j-1$ 欄消去後剩餘 Schur complement 的第一個對角元素。SPD 的主區塊和 Schur complement 都是 SPD，故此對角元素嚴格為正；因此 $l_{jj} > 0$，欄內的除法有定義。

逐欄代回 $a_{ij} = \sum_{k \le \min(i,j)} l_{ik} l_{jk}$，可見每一步構造的元素都使對應矩陣條目相等，最後 $A = LL^T$。相同的正對角條件迫使每一步的根號及同欄元素唯一，因此分解唯一。若輸入並非實對稱正定，不能保證根號為正或除數非零。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 用遞迴式逐項算同一個 3×3 例子

對 $A=[4,2,2;2,5,1;2,1,3]$，第一欄：$l_{11}=\sqrt{4}=2$，$l_{21}=2/2=1$，$l_{31}=2/2=1$。

第二欄：$l_{22}=\sqrt{5-l_{21}^2}=2$，$l_{32}=(1-l_{31}l_{21})/l_{22}=0$。第三欄：$l_{33}=\sqrt{3-l_{31}^2-l_{32}^2}=\sqrt{2}$。

所以 $L=[2,0,0;1,2,0;1,0,\sqrt{2}]$；核對 $l_{31}l_{21}+l_{32}l_{22}=1=a_{32}$，並以 $LL^T=A$ 驗算所有元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Cholesky 算例：每次更新都看得見

沿用 $A=\begin{bmatrix}4&2&2\\2&5&1\\2&1&3\end{bmatrix}$。第一步 $\ell_{11}=2$，其下兩數都是 $1$，尾塊更新為 $\begin{bmatrix}5&1\\1&3\end{bmatrix}-\begin{bmatrix}1&1\\1&1\end{bmatrix}=\begin{bmatrix}4&0\\0&2\end{bmatrix}$。

因此 $L=\begin{bmatrix}2&0&0\\1&2&0\\1&0&\sqrt{2}\end{bmatrix}$。相乘 $LL^T$ 回到 $A$；對任意非零 $x$，$x^TAx=\|L^Tx\|^2>0$，也驗證本例正定。



## 已知 Cholesky，怎麼解方程？

$LL^Tx=b$ 分成 $Ly=b$ 的前代與 $L^Tx=y$ 的回代。取 $b=(8,8,6)^T$：前代得 $y=(4,2,\sqrt{2})^T$；回代依序得 $x_3=1$、$x_2=1$、$x_1=1$。代回可確認 $A(1,1,1)^T=b$。

一般稠密矩陣的 Cholesky 分解需 $\Theta(n^3)$ 算術運算；每個右端的兩次三角解為 $\Theta(n^2)$。SPD 是必要使用前提，浮點輸入近似正定時仍需處理捨入與失敗。

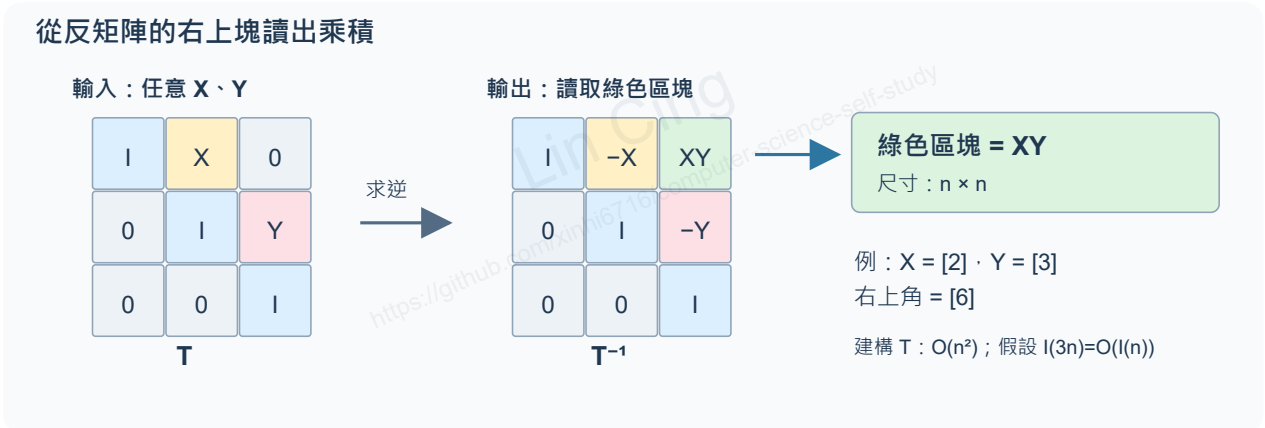
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 反矩陣可以算出矩陣乘積

欲計算兩個 $n\times n$ 矩陣 $X,Y$ 的乘積，建立上三角分塊矩陣 $T=\begin{bmatrix} I & X & 0 \\ 0 & I & Y \\ 0 & 0 & I \end{bmatrix}$。

$T^{-1}=\begin{bmatrix} I & -X & XY \\ 0 & I & -Y \\ 0 & 0 & I \end{bmatrix}$：直接做分塊乘法，右上角 $XY-XY=0$，其餘非對角塊也抵消。讀出右上塊便得到 $XY$；$T$ 對任意 $X,Y$ 均可逆。




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 從反矩陣讀出乘積：成本條件

若 $I(n)$ 是 $n \times n$ 可逆矩陣的反矩陣演算法上界，且 $I(n) = \Omega(n^2)$、$I(3n) = O(I(n))$，建立 $3n \times 3n$ 的 $T$ 並讀取右上塊只需 $O(n^2)$，故矩陣乘法成本 $M(n) = O(I(n))$。

這是算術操作模型的歸約。倍增規律不可省略；若只知某個規模的反矩陣成本，不能直接把 $I(3n)$ 換成 $I(n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 從快速乘法到快速反矩陣

先討論 SPD 矩陣。分塊公式需要遞迴求 $B^{-1}$ 和 $S^{-1}$，以及常數次半尺寸乘法與 $O(n^2)$ 次加減，故 $I_{SPD}(n) \le 2I_{SPD}(n/2) + O(M(n) + n^2)$。

若 $M(n) = \Omega(n^2)$、$M(n/2) \le cM(n)$ 對某固定 $c < 1/2$ 成立，則各層總成本以比率至多 $2c < 1$ 遞減，總計 $I_{SPD}(n) = O(M(n))$。這不是所有任意形狀函數 $M$ 無條件成立的結論。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 一般可逆矩陣與算術模型限制

對實數可逆 $A$，$A^TA$ 是 SPD：$x^TA^TAx = \|Ax\|^2 > 0$。先形成並求逆 $A^TA$，再乘 $A^T$，即 $A^{-1} = (A^TA)^{-1}A^T$。若補零至二次冪，使用 $\mathrm{diag}(A, I)$；還需假設 $M(n+k) = O(M(n))$（$0 \le k \le n$）。

在上述正規性及精確算術成本模型下，乘法與求逆互相歸約。這不是浮點穩定性的保證：$A^TA$ 的條件數會平方，實務求解通常採用有主元消去或適當分解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## LDLᵀ 與 Cholesky 的關係

對 SPD 矩陣，也可把消去的主元留在對角矩陣：$A = LDL^T$，其中 $L$ 為對角線全 1 的下三角矩陣，$D$ 為正對角矩陣。每一步的主元是目前 SPD Schur complement 的左上角，故必為正。

若 $D = \mathrm{diag}(d_1, \dots, d_n)$，設 $D^{1/2} = \mathrm{diag}(\sqrt{d_i})$，則 $A = (LD^{1/2})(LD^{1/2})^T$，這就是正對角 Cholesky 因子。前例 $L = [1,0,0;\, 1/2,1,0;\, 1/2,0,1]$、$D = \mathrm{diag}(4,4,2)$；乘 $D^{1/2}$ 得 $[2,0,0;\, 1,2,0;\, 1,0,\sqrt{2}]$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 稠密分解的主導運算與換列成本

假設稠密 $n \times n$、標量加減乘除各算 1 次 flop，且無提前奇異停止。LUP 的尾塊更新以乘法加減法計約 $2\sum_{s=1}^{n-1} s^2 = \frac{2}{3}n^3 + O(n^2)$ 次 flop；乘數除法僅 $O(n^2)$。一個右端的前代加回代共約 $2n^2 + O(n)$ 次 flop。

Cholesky 的逐元素乘加約 $\frac{1}{3}n^3 + O(n^2)$ 次 flop；除法與平方根分別為 $O(n^2)$ 和 $n$ 次，若平方根成本不同應另列。這兩個領先係數適用所述 flop 慣例，不是執行時間比例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 選主元、記憶體及數值成本另計

LUP 的 partial pivot 每欄掃描絕對值並比較，合計 $O(n^2)$ 次比較；至多 $n$ 次整列交換，最多移動 $O(n^2)$ 個矩陣元素及 $O(n)$ 個排列索引。這些比較、記憶體讀寫與平方根通常不放進上述乘加 flop，實際執行時間可能由記憶體或實作決定。

工作矩陣 $W$ 使用 $\Theta(n^2)$ 儲存，額外排列向量 $\Theta(n)$；覆寫與否及保存原矩陣會改變額外空間。稀疏矩陣、低秩捷徑、平行化與提早停止不適用上述稠密完整流程的直接時間結論。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 三個不共線點無法全數插值

資料 $(t,b)=(0,1),(1,2),(2,2)$，擬合 $b\approx\alpha+\beta t$。$A=\begin{bmatrix}1&0\\1&1\\1&2\end{bmatrix}\in\mathbb{R}^{3\times 2}$，$x=(\alpha,\beta)^T$，$b=(1,2,2)^T$。相鄰斜率為 1 與 0，故沒有一直線通過三點。目標固定為 $\min_x \|b-Ax\|_2^2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 正規方程把導數設為零

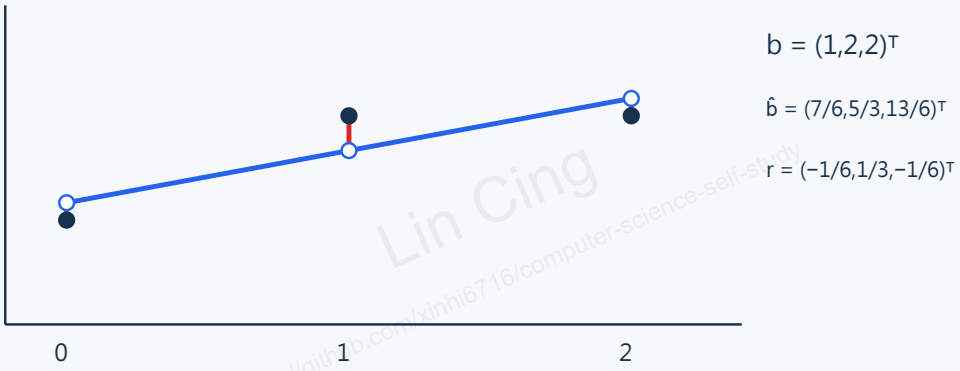
$\nabla\|b-Ax\|^2=-2A^T(b-Ax)=0$，故 $A^TA\hat{x}=A^Tb$。本例 $A^TA=\begin{bmatrix}3&3\\3&5\end{bmatrix}$，$A^Tb=(5,6)^T$。$A$ 滿欄秩，$z^TA^TAz=\|Az\|^2>0$ 對所有非零 $z$ 成立，故解唯一。若欄相依，這個唯一性結論不成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 解出係數再逐點算預測與殘差

正規方程為 $3\alpha + 3\beta = 5$、$3\alpha + 5\beta = 6$。相減得 $\beta = 1/2$，再得 $\alpha = 7/6$。預測 $A\hat{x} = (7/6, 5/3, 13/6)^T$，殘差 $r = (-1/6, 1/3, -1/6)^T$。平方和為 $1/36 + 1/9 + 1/36 = 1/6$。

### 垂直殘差，不是最短點線距離



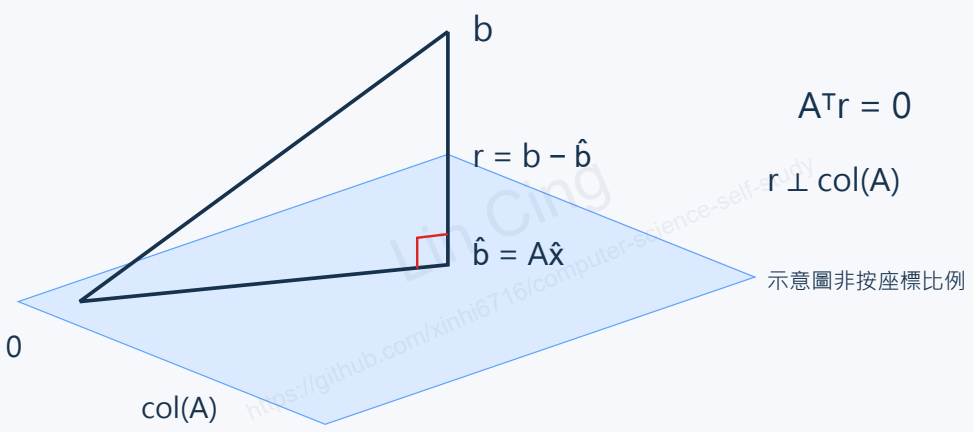

紅線固定 t；藍線 b̂ = 7/6 + t/2



## 垂直殘差與資料空間正交是兩種圖

散點圖在固定 $t_i$ 上畫垂直線段，長度 $|b_i - \hat{b}_i|$。它不是點到直線的最短距離。在 $\mathbb{R}^3$ 資料空間，$\hat{b} = A\hat{x} \in \text{col}(A)$，$r \perp \text{col}(A)$。直接檢查 $A^T r = (-1/6 + 1/3 - 1/6,\ 1/3 - 2/6)^T = (0, 0)^T$。

### 資料空間 R³ 的正交投影示意



## 正規方程會平方條件數

滿欄秩時 $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$，而 $A^TA$ 特徵值為 $\sigma_i^2$，所以 $\kappa_2(A^TA) = \kappa_2(A)^2$。例如 $A = \mathrm{diag}(1, 10^{-4})$ 的條件數為 $10^4$，$A^TA$ 為 $10^8$。浮點形成 $A^TA$ 會使弱方向更難保留；不是每題都固定損失相同位數。






## QR 避免先形成平方條件數的矩陣

薄 QR 為 $A = QR$、$Q^TQ = I$。最小平方改解 $R\hat{x} = Q^Tb$。本例 $q_1 = (1,1,1)^T/\sqrt{3}$、$q_2 = (-1,0,1)^T/\sqrt{2}$，$R = [\sqrt{3}, \sqrt{3}; 0, \sqrt{2}]$，$Q^Tb = (5/\sqrt{3}, 1/\sqrt{2})^T$，仍得 $(7/6, 1/2)^T$。實務用 Householder QR；秩疑慮用 pivoted QR 或 SVD。QR 不會消除資料本身病態。

## 一般 rank：正規方程是必要條件

令 $f(x) = \|b - Ax\|_2^2$。若 $\hat{x}$ 是全域最小解，對任意方向 $z$，一元函數 $g(t) = f(\hat{x} + tz)$ 在 $t = 0$ 達到最小值，故 $g'(0) = -2z^T A^T (b - A\hat{x}) = 0$。

此式對所有 $z$ 成立，因此 $A^T(b - A\hat{x}) = 0$，也就是 $A^T A\hat{x} = A^T b$。推導不要求 $A$ 滿欄秩或 $A^T A$ 可逆。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 一般 rank：正規方程也是充分條件

反過來，若 $\hat{x}$ 滿足 $A^T(b - A\hat{x}) = 0$，設 $r = b - A\hat{x}$。對任意 $z$ 展開：

$\|b - A(\hat{x} + z)\|^2 = \|r - Az\|^2 = \|r\|^2 - 2z^T A^T r + \|Az\|^2 = \|r\|^2 + \|Az\|^2 \geq \|r\|^2$。

故 $\hat{x}$ 為全域最小解；等號恰在 $Az = 0$，所以全部解恰為 $\hat{x} + \ker(A)$。又 $A\hat{x} \in \mathrm{col}(A)$ 且 $r \perp \mathrm{col}(A)$，故它就是 $b$ 的唯一正交投影。有限維欄空間的投影存在，因而至少有一個 $\hat{x}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 滿欄秩時的投影矩陣

若 $A \in \mathbb{R}^{m \times n}$ 滿欄秩，$A^T A$ 為 SPD，$\hat{x} = (A^T A)^{-1} A^T b$ 是唯一最小平方解。定義 $P = A(A^T A)^{-1} A^T$，則預測 $p = Pb$。

由 $P^T = P$ 與 $P^2 = P$，$P$ 是到 $\mathrm{col}(A)$ 的正交投影；$A^T(b - Pb) = 0$ 直接給出殘差正交。欄不滿秩時不能使用 $(A^T A)^{-1}$，應採上一節的 SVD／廣義逆表示。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 欄不獨立時：預測唯一，係數可能不唯一

對任意實矩陣 $A$ 與向量 $b$，最小平方問題 $\min_x \|Ax - b\|_2^2$ 一定有解；所有解都滿足 $A^T Ax = A^T b$。原因是 $b$ 在 $\mathrm{col}(A)$ 的正交投影 $p$ 存在且唯一；任何滿足 $Ax = p$ 的 $x$ 都是解。

若 $A$ 的欄相依，$\ker(A)$ 含非零向量。只要 $x_0$ 是解，全部解恰為 $x_0 + \ker(A)$；因此 $Ax = p$ 仍唯一，但係數不唯一。$A = 0$ 時所有 $x$ 均達到相同殘差。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 欄不獨立的具體算例

取 $A=\begin{bmatrix}1&1\\2&2\\0&0\end{bmatrix}$、$b=(2,3,1)^T$，令 $u=(1,2,0)^T$。$p=u(u^Tb)/(u^Tu)=(8/5,16/5,0)^T$，$r=b-p=(2/5,-1/5,1)^T$，可驗 $A^Tr=0$。

$Ax=p$ 等價於 $x_1+x_2=8/5$，故所有解為 $(4/5,4/5)^T+t(1,-1)^T$。這些係數給出相同預測；其中 $t=0$ 的範數最小，為 $(4/5,4/5)^T$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## SVD 選出唯一的最小範數解

設 $A=U\Sigma V^T$ 的非零奇異值為 $\sigma_1,\dots,\sigma_r$，定義 $A^+=\sum_{i=1}^{r}\sigma_i^{-1}v_iu_i^T$。$x_*=A^+b$ 是唯一的最小範數最小平方解；全部最小平方解為 $x_*+z$，其中 $z\in\ker(A)$。

由 $x_*\in\ker(A)^\perp$，$\|x_*+z\|^2=\|x_*\|^2+\|z\|^2$，所以 $z=0$ 唯一最小。秩為零時空和為零，$A^+=0$；數值實作須依資料尺度選擇秩截斷閾值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 為何要計算多項式的點值？

長度 $m$ 與 $n$ 的係數向量直接相乘需 $O(mn)$ 次算術運算。先在共同點求值，逐點相乘，再內插回係數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 點值能否唯一還原係數？

取域 $F$ 中 $n$ 個點 $x_0, \dots, x_{n-1}$，係數向量 $a$ 與點值 $y$ 滿足 $y = Va$，其中 $V_{ij} = x_i^j$（$0 \le i, j < n$）。這個線性系統何時有唯一解？關鍵是以下行列式，而不是僅憑「方陣」判斷。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Vandermonde 行列式

$\det V = \prod_{0 \le i < j < n} (x_j - x_i)$。因此在域上，點兩兩相異當且僅當 $V$ 可逆；若兩個點相同，對應兩列相同。

例：點 $1, 2, 4$ 的行列式為 $(2-1)(4-1)(4-2) = 6$。在實數域它非零；在 $\mathbb{F}_3$ 中 $1$ 與 $4$ 是同一個點，行列式為零。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 證明：把最後一個點看成變數

對 $n$ 歸納。$n = 1$ 時行列式和空乘積都是 $1$。先把 $x_0, \dots, x_{n-2}$ 視為互異的不定元，令 $D(t)$ 是最後一列為 $[1, t, \dots, t^{n-1}]$ 的行列式。

$D(t)$ 是次數至多 $n-1$ 的多項式。當 $t = x_i$ 時有兩列相同，故 $D(x_i) = 0$；在多項式環 $F[x_0, \dots, x_{n-2}, t]$ 中，這些兩兩不同的 monic 一次因子皆為不可約元素，因此它們的乘積整除 $D(t)$，得到 $D(t) = c \prod_{i=0}^{n-2} (t - x_i)$。最後可將不定元代入任意域元素，即使代入後有重複點也成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 證明：決定最高次係數

沿最後一列展開，唯一含 $t^{n-1}$ 的項是右下角項，其餘子式是前 $n-1$ 個點的 Vandermonde 行列式，符號為正。

因此 $c = \det V_{n-1} = \prod_{0 \le i < j < n-1} (x_j - x_i)$。代回 $t = x_{n-1}$ 即得所需乘積，歸納完成。此證明也涵蓋重複點，無須先假設各點不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Lagrange 插值：不只知道解存在

若 $x_0, \dots, x_{n-1}$ 兩兩相異，定義 $L_i(z) = \prod_{j \ne i} \frac{z - x_j}{x_i - x_j}$。每個分母在域中非零，因此可逆；$L_i(x_k) = 1$ 當 $i = k$，否則為 $0$。

給定任意點值 $y_i$，$P(z) = \sum_i y_i L_i(z)$ 的次數小於 $n$ 且 $P(x_i) = y_i$，這直接構造解。若另一個次數小於 $n$ 的 $Q$ 有相同點值，則 $P - Q$ 有 $n$ 個不同根，只能是零多項式；故插值唯一。這也與 Vandermonde 可逆結論一致。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 如何在 $O(n^2)$ 求插值多項式的係數？

先逐因子相乘求 $G(z)=\prod_{j=0}^{n-1}(z-x_j)$，第 $k$ 次更新長度 $k+1$ 的係數列，總共 $O(n^2)$ 次域運算。

對每個 $i$，以 synthetic division（綜合除法）用 $O(n)$ 算出 $Q_i(z)=G(z)/(z-x_i)$；以 Horner 法在 $x_i$ 評估 $d_i=Q_i(x_i)=\prod_{j\neq i}(x_i-x_j)\neq 0$。再將 $y_i d_i^{-1}Q_i$ 的 $n$ 個係數加到答案。$n$ 次各花 $O(n)$，所以總共 $O(n^2)$ 次域運算；假設域的四則運算是單位成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 為什麼少於 $n$ 個點不能唯一決定？

給定 $m<n$ 個互異點 $x_0,\dots,x_{m-1}$，零多項式 $P(z)=0$ 和 $Q(z)=\prod_{i=0}^{m-1}(z-x_i)$ 都在這些點取值零，卻是不同多項式，且次數都小於 $n$。

例如要決定二次以內的多項式，只知道 $P(0)=P(1)=0$，則 $P(z)=0$ 和 $P(z)=z(z-1)$ 都符合。再多一個不同點的值，才有機會利用前述 $n$ 點唯一性定理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Horner 法：單點求值的不變量

對 $A(z)=\sum_{j=0}^{n-1}a_jz^j$，固定求值點 $x$，令 $b=0$；依 $k=n-1,n-2,\dots,0$ 更新 $b\leftarrow a_k+xb$。完成第 $k$ 次後，迴圈不變量是 $b=\sum_{j=k}^{n-1}a_jx^{j-k}$。初始化時 $b=0$ 對空和成立；每次更新把冪次多乘一個 $x$ 並加入 $a_k$；結束時 $k=0$，得到 $A(x)$。

例如 $A(z)=2+3z+4z^2$、$x=2$：$b$ 依序為 $4$、$3+2\cdot4=11$、$2+2\cdot11=24$。此實作對 $n$ 個係數使用 $\Theta(n)$ 次域運算；對 $n$ 個點逐一重做 Horner 則為 $\Theta(n^2)$，是此方法的成本，不是所有多點求值方法的下界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 固定變換慣例

令 $n=2^s$、$\omega_n=e^{2\pi i/n}$，定義 $X_k=\sum_{j=0}^{n-1}x_j\omega_n^{jk}$。此處正向 DFT 使用正指數；索引皆為 $0$ 至 $n-1$。

$\omega_4 = i$

k: 0 1 2 3

$\omega_4^k$: 1 i −1 −i

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 單位根與週期

偶數 $n$ 下，$\omega_n^n = 1$、$\omega_n^{k+n} = \omega_n^k$，且 $\omega_n^{k+n/2} = -\omega_n^k$。$n = 4$ 時依序為 $1, i, -1, -i$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 拆成偶數與奇數係數

令 $A(z) = E(z^2) + zO(z^2)$，其中 $E(t) = \sum_r x_{2r} t^r$，$O(t) = \sum_r x_{2r+1} t^r$。兩個半長 DFT 共用平方後的單位根。

A(z) = E(z²) + z O(z²)

E: x₀, x₂, x₄, …

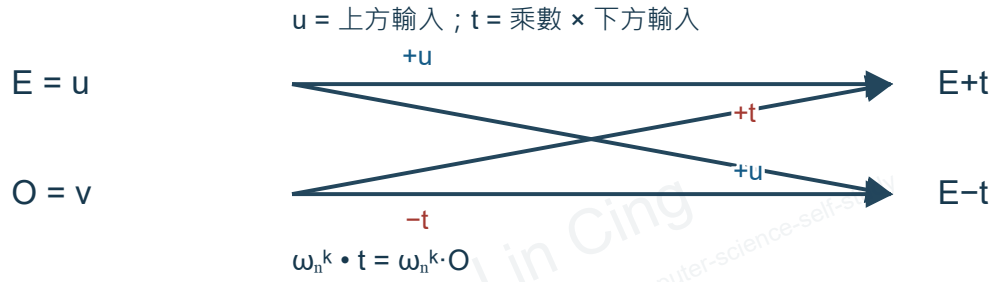
O: x₁, x₃, x₅, …

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 合併一對輸出

對 $0 \le k < n/2$，令 $t = \omega_n^k O_k$，則 $X_k = E_k + t$、$X_{k+n/2} = E_k - t$。這一對運算稱蝶形，乘數稱旋轉因子。

### 一個蝶形




Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 為何成本是 $O(n \log n)$

$T(n) = 2T(n/2) + O(n)$ 有 $\log_2 n$ 層，每層 $O(n)$。原地迭代版每階段有 $n/2$ 個蝶形，共 $\log_2 n$ 階。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 輸入索引做位元反轉

$n = 4$ 時，兩位元索引 $00, 01, 10, 11$ 反轉為 $00, 10, 01, 11$。因此 $[1, 2, 0, -1]$ 重排為 $[1, 0, 2, -1]$，供時間抽取的迭代階段使用。

索引： 00 01 10 11

反轉： 00 10 01 11

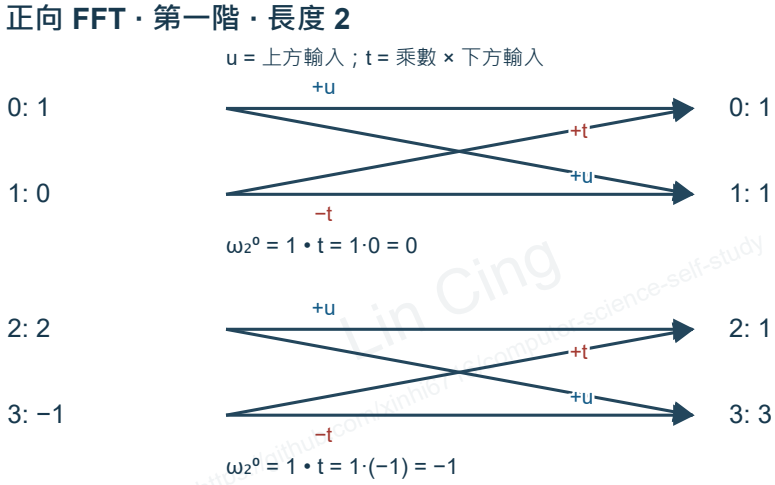
數值： 1 0 2 −1

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 第一階，長度 2

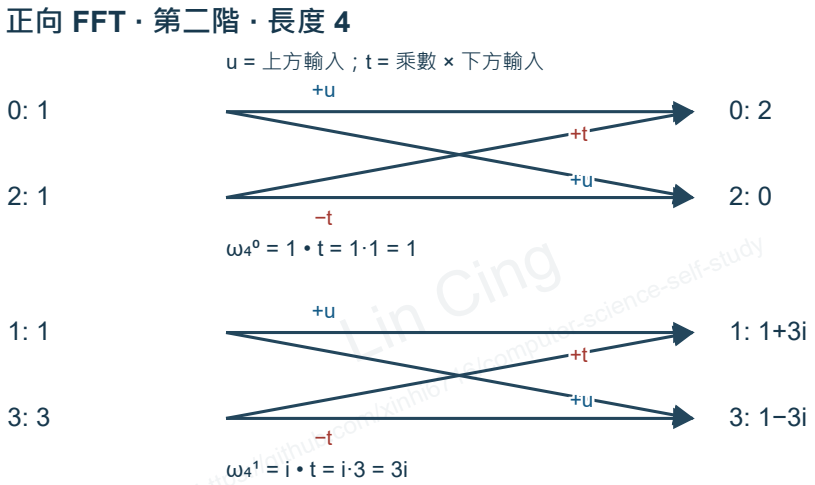
輸入 $[1, 0, 2, -1]$，兩個乘數都是 $\omega_2^0 = 1$。索引 $(0, 1)$ 得 $(1 + 0, 1 - 0) = (1, 1)$；$(2, 3)$ 得 $(2 - 1, 2 - (-1)) = (1, 3)$。狀態為 $[1, 1, 1, 3]$。




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 第二階，長度 4

輸入 $[1,1,1,3]$。$(0,2)$ 用 $\omega_4^0=1$，得 $(2,0)$；$(1,3)$ 用 $\omega_4^1=i$，得 $(1+3i,1-3i)$。按索引排列 $X=[2,1+3i,0,1-3i]$。






## 用定義直接核對

$X_0=1+2+0-1=2$；$X_1=1+2i+(-1)(-i)=1+3i$；$X_2=1-2+0+1=0$；$X_3=1-2i+(-1)i=1-3i$。此計算獨立核對索引與符號。

## 迭代蝶形：每一階完成了什麼？

令 $N = 2^{\ell}$，第 $s$ 階（$0 \le s \le \ell$）完成後，每個連續長度 $2^s$ 的區塊都保存一個長度 $2^s$ 的 DFT，依自然頻率順序排列。對第 $b$ 個區塊，輸入原序列為 $x_{r+tN/2^s}$，其中 $0 \le t < 2^s$，$r$ 是把 $b$ 的 $\ell - s$ 位元反轉得到的值。




框內列出 DFT 所處理的原序列索引；輸出在各框內依自然頻率排列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 不變量如何由蝶形維持？

$s = 0$ 時，每個單元素區塊就是原始資料的位元反轉排列，不變量成立。設某階已成立：下一階每個區塊的前、後半恰好分別是同一子序列偶數及奇數位置的 DFT。對每個 $k$，用 $E_k \pm \omega_{2^{s+1}}^{k} O_k$ 組成位置 $k$ 與 $k + 2^s$ 的點值，故新區塊成為完整 DFT。

兩個半區塊的原索引間隔均為 $N/2^s$，其起始索引差為 $N/2^{s+1}$，正是新子序列的偶數／奇數位置。歸納至 $s = \ell$，得到整個輸入的自然頻率順序 DFT。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 位元反轉不是免費操作

若 $N = 2^{\ell}$ 且 $i$ 以 $\ell$ 位元表示，$\mathrm{rev}_{\ell}(i)$ 是其反向位元所代表的索引。最直接的方法逐一掃描每個索引的 $\ell$ 位元，總工作量 $O(N \log N)$。若採用反向計數器，每次更新只翻轉受進位影響的位元，所有 $N$ 次更新合計 $O(N)$ 次位元翻轉，可用 $O(N)$ 工作產生全部索引。

原地重排只在 $i < \mathrm{rev}_{\ell}(i)$ 時交換，以免交換兩次。前面平行 FFT 的 $O(1)$ 重排深度假設映射已提供；若映射要自己生成，其成本必須另計。不同字長和指令模型也會影響位元操作成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 迭代演算法

先位元反轉。依序令 $L = 2, 4, \dots, n$；對每個起點為 $b$ 的區塊及 $0 \le j < L/2$，令 $u = a_{b+j}$、$t = \omega_L^j a_{b+j+L/2}$，再同時寫入 $(u + t, u - t)$。寫入前先保存 $u, t$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 逆變換

$x_j = \frac{1}{n}\sum_{k=0}^{n-1} X_k \omega_n^{-jk}$。以逆單位根執行同一 FFT，再將每項除以 $n$。

位元反轉 → 逆根 FFT → 除以 n

[2, 1+3i, 0, 1−3i] → [1, 2, 0, −1]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



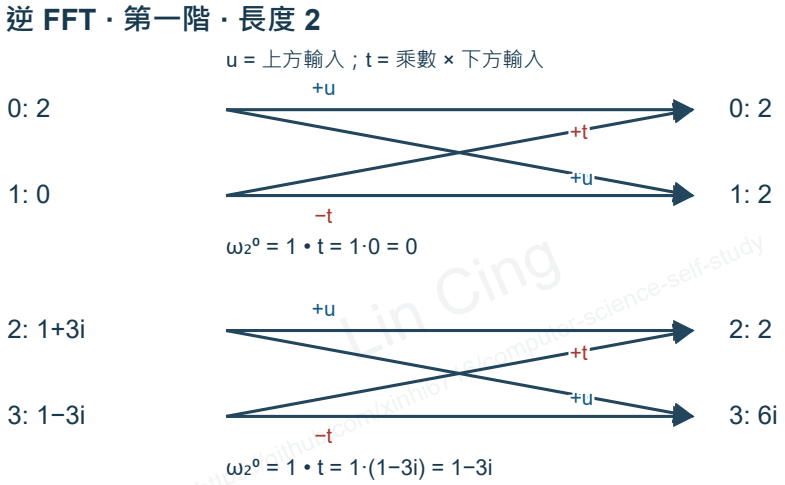
## 為何要除以 $n$

代入 $X_k$ 得 $\frac{1}{n}\sum_r x_r \sum_k \omega_n^{k(r-j)}$。因 $0 \le r, j < n$，內層等比級數在 $r = j$ 時為 $n$，其他時為 $0$，故只剩 $x_j$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 逆變換例子，第一階

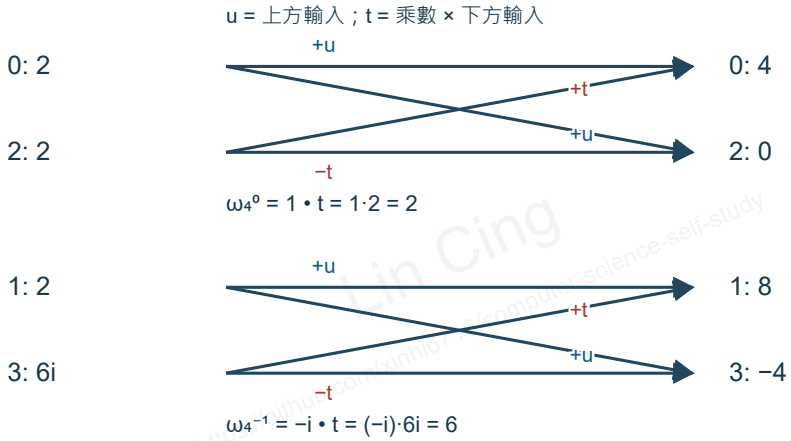
將 $[2, 1+3i, 0, 1-3i]$ 位元反轉為 $[2, 0, 1+3i, 1-3i]$。長度 2 的乘數皆為 $1$，兩對運算得到 $[2, 2, 2, 6i]$。






## 逆變換例子，第二階

使用 $\omega_4^{-1} = -i$。$(0, 2)$ 的乘數為 $1$，結果 $(4, 0)$；$(1, 3)$ 的乘積為 $(-i)(6i) = 6$，結果 $(8, -4)$。將 $[4, 8, 0, -4]$ 除以 $4$ 得 $[1, 2, 0, -1]$。



## 遞迴 FFT 的空間：看同時活著的資料

順序深度優先實作若每層配置長度與該子問題成正比的暫存，並在完成後釋放，沿單條遞迴路徑同時存活的容量至多 $N + N/2 + N/4 + \cdots < 2N$；加上輸入／輸出和 $O(\log N)$ 呼叫堆疊，峰值額外空間為 $O(N)$。

若有意保留每一階全部節點的陣列供追蹤或重用，每階合計 $N$ 個元素，共 $\log_2 N + 1$ 階，儲存量則為 $\Theta(N \log N)$。兩者的運算量都可為 $\Theta(N \log N)$；空間差異來自保留策略，非單純由遞迴層數決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 為什麼正向 DFT 做兩遍不等於逆變換？

在正向慣例 $X_k = \sum_j x_j \omega^{jk}$ 下，再做一次正向變換並除以 $N$，可交換求和得到 $\frac{1}{N}\sum_k X_k \omega^{k\ell} = \frac{1}{N}\sum_j x_j \sum_k \omega^{k(j+\ell)} = x_{(-\ell) \bmod N}$。

內層和僅在 $j + \ell \equiv 0 \pmod N$ 時為 $N$。所以結果是循環索引反轉；逆變換須使用 $\omega^{-1}$ 再除以 $N$，或正向做第二次後再反轉索引。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 一個實際反例

令 $N=4$、$\omega=i$、$x=[1,2,0,0]$。一次正向 DFT 為 $[3,1+2i,-1,1-2i]$；第二次仍採正向根，再除以 $4$，得到 $[1,0,0,2]$，不是原輸入 $[1,2,0,0]$。錯誤不是少除一個 $N$，而是頻率指數方向不對；改用逆根 $-i$ 即可還原。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 平行 FFT 的工作量與深度

以同步、隨機存取的 EREW PRAM 為模型，假設 $N=2^s\ge 2$、位元反轉位址映射已給定，每階段可以配置 $N/2$ 個處理器。每個處理器讀一對獨立輸入、算一個蝶形並寫回該對；不同蝶形不讀寫同一格，階段之間有同步屏障。旋轉因子可預先給定。

位元反轉搬移用 $O(N)$ 工作與 $O(1)$ 時間；每一階用 $\Theta(N)$ 工作、$\Theta(1)$ 時間，共 $\log_2 N$ 階。因此 $W=\Theta(N\log N)$、深度 $D=\Theta(\log N)$，額外儲存 $O(N)$。這不是宣稱任何實機均可達到；若生成索引或旋轉因子也需計費，必須另加其成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 線性與循環卷積

長度 $N$ 的變換逐點相乘得到循環卷積 $c_k = \sum_{j=0}^{N-1} a_j b_{(k-j) \bmod N}$。補零不足時，高次係數會回繞並加到前面。

m + n − 1 ≤ N

[1, 2, 0, 0] × [3, 4, 0, 0]

線性輸出: [3, 10, 8, 0]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 先補零再相乘

對長度 $m, n \geq 1$，選取二的冪 $N \geq m + n - 1$，兩向量補零至 $N$，FFT 後逐點相乘，再做逆 FFT。前 $m + n - 1$ 項即線性卷積。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 卷積例子

$[1,2] * [3,4] = [3,10,8]$。取 $N=4$，補成 $[1,2,0,0]$、$[3,4,0,0]$，逐點乘積的逆變換為 $[3,10,8,0]$。若取 $N=2$，係數 $8$ 會回繞至常數項。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 複數 FFT 卷積：先做兩次正轉換

以 $A(z)=1+2z$、$B(z)=3+4z$ 為例；線性卷積長度為 $2+2-1=3$，因此補零至 $N=4$。採正向 $\omega=i$：$A=[1,2,0,0]$、$B=[3,4,0,0]$。

由 $X_k=\sum_j x_j i^{jk}$，依序在 $1, i, -1, -i$ 求值，得到 $\widehat{A}=[3,1+2i,-1,1-2i]$ 和 $\widehat{B}=[7,3+4i,-1,3-4i]$。例如 $A(i)=1+2i$、$B(-1)=3-4=-1$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 逐頻相乘，再算逆變換

逐點乘法得 $C = [21, -5+10i, 1, -5-10i]$；例如 $(1+2i)(3+4i) = -5+10i$。逆變換採 $i^{-1} = -i$，每個輸出是 $c_j = \frac{1}{4}\sum_{k=0}^{3} C_k(-i)^{jk}$。

四個未除以 4 的和依序為 $[12, 40, 32, 0]$，因此 $c = [3, 10, 8, 0]$。這與直接相乘 $(1+2z)(3+4z) = 3+10z+8z^2$ 一致。圖中依同一個頻率相乘，不能交叉配對。

| 頻率 | A 的點值 | B 的點值 | | 逐頻相乘 |
|---|---|---|---|---|
| 1 | 3 | 7 | × | 21 |
| i | 1+2i | 3+4i | × | −5+10i |
| −1 | −1 | −1 | × | 1 |
| −i | 1−2i | 3−4i | × | −5−10i |

逆變換： [21, −5+10i, 1, −5−10i] → [3, 10, 8, 0]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 有限域版 Fourier 變換

選質數 $p$，令 $N = 2^s$ 且 $N \mid (p-1)$。選 $\omega \in \mathbb{F}_p$，使它的乘法階恰為 $N$：$\omega^N = 1$，而 $1 \le k < N$ 時 $\omega^k \neq 1$。有限域非零元素形成階 $p-1$ 的循環群，故此條件足以保證存在。

定義 $X_k = \sum_{j=0}^{N-1} x_j \omega^{jk} \pmod p$。所有乘加都在 $\mathbb{F}_p$ 中計算；複數的 $i$ 在此由合適的 $\omega$ 取代。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 為什麼逆變換有效？

因 $N \mid p-1$，所以 $p \nmid N$，$N^{-1}$ 存在。對 $0 < r < N$，$\omega^r \neq 1$，且 $1-\omega^r$ 在域中可逆，故幾何和 $\sum_{k=0}^{N-1} \omega^{kr} = 0$；$r = 0$ 時其值為 $N$。

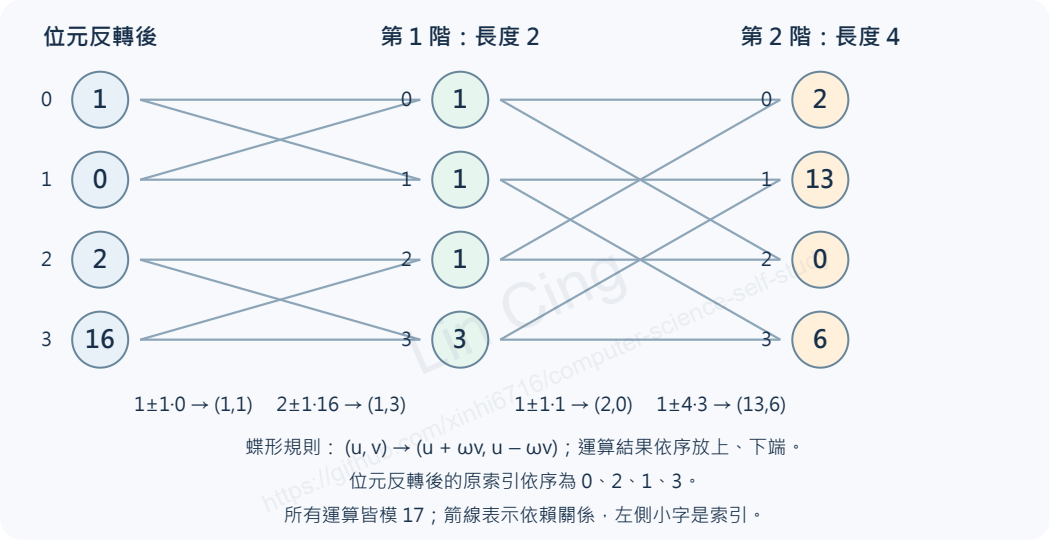
代入並交換求和可得 $x_j = N^{-1} \sum_{k=0}^{N-1} X_k \omega^{-jk} \pmod{p}$。這個論證仰賴域中非零元素可逆；不能把它直接推廣到任意合數模數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 具體 NTT：模 17 的兩層蝶形

$N = 4$、$p = 17$、$\omega = 4$：$4^2 = 16 = -1$、$4^4 = 1$，所以階恰為 4；$4^{-1} = 13$。輸入 $[1, 2, 0, -1]$，在模 17 中為 $[1, 2, 0, 16]$。




依一般二次求值也能核對輸出 $[2, 13, 0, 6]$。每個負數都先按模 17 解讀。

## 點乘後的結果與整數還原

對 $A=[1,2]$、$B=[3,4]$，補零到 $N=4$。它們的 NTT 分別為 $[3,9,16,10]$、$[7,2,16,4]$；逐點乘得 $[4,1,1,6]$，逆變換為 $[3,10,8,0]$，對應 $3+10z+8z^2$。

要得到線性卷積而非循環卷積，須 $N \geq r+s-1$。對整數輸入，模 $p$ 結果只決定餘數；若已知每格係數絕對值至多 $M$ 且 $p > 2M$，才可從對稱代表元唯一還原。此例 $r=s=2$、$|a_i|, |b_i| \leq 4$，取 $M = 2 \cdot 4^2 = 32$，而 $17 \not> 64$；例如餘數 $10$ 也代表整數 $-7$；若未另給足夠的範圍資訊，不能從模數結果判定實際係數是 10 還是 -7。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 有號整數卷積：界限足夠的例子

令 $A=[1,2]$、$B=[1,-1]$，仍用 $p=17$、$N=4$、$\omega=4$。補零後，兩個 NTT 分別是 $[3,9,16,10]$、$[0,14,2,5]$，逐點相乘為 $[0,7,15,16]$；逆變換得到 $[1,1,15,0]$。

因輸入兩邊係數的絕對值均不超過 $B=2$，每個卷積係數至多含 $\min(2,2)=2$ 項乘積，故 $M = 2B^2 = 8$。$p=17 > 2M=16$，其對稱代表元落在 $[-8,8]$：$15$ 還原為 $-2$，因而得到 $[1,1,-2,0]$。

長度 $N$ 的 FFT 需 $\Theta(N \log N)$ 次模 $p$ 算術操作；若 $p$ 隨輸入大小增長，模乘、模加的成本還取決於 $\log p$ 位元的運算方法，不能一律當成固定時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 精度與成本

三次變換與 $N$ 次逐點乘法需 $O(N \log N)$ 次算術運算及 $O(N)$ 空間。這不是位元複雜度保證。浮點誤差會累積；只有證明每項絕對誤差小於 $1/2$ 才能安全四捨五入為整數，否則須提高精度或採精確方法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 選擇適合的方法

小輸入直接乘法可能較簡單。FFT 需要足夠補零與數值誤差預算；精確整數結果可考慮合適模數的數論變換與有界重建。介面應核對符號、索引和正規化。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料與署名

CLRS, Introduction to Algorithms, third edition, chapters 28 and 30. Matrix: PDF pages 834–861. FFT: PDF pages 919, 921, 923, 928–936, 939–941, 944–945 (PDF page numbering, third edition). MIT 6.046J Fall 2004 lecture 3 slides, PDF pages 27 and 35–43, and Fall 2005 lecture 3 notes support matrix multiplication and Strassen, not FFT.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study